Sistema de Planificación de Viajes en Transporte Público con Validación Formal en VDM++ para la Ciudad de Arequipa

Alvarez Choque Miguel Angel, Cuno Salazar Eduardo Joel, Cruz Gallegos Jeremy Jeison,
Inca Moncca Jherson David, Quispe Madariaga Jeferson Jofre
Universidad Nacional de San Agustín, Arequipa, Perú

***Resumen*—La información sobre las rutas de transporte público en Arequipa es dispersa e imprecisa, lo que provoca desplazamientos ineficientes y decisiones de movilidad erróneas. Este trabajo propone una plataforma web para la búsqueda de rutas directas y combinadas, cuya componente crítica, el algoritmo de planificación de viajes, es especificada y validada formalmente con el lenguaje VDM++. Se definen los tipos de datos e invariantes del dominio de rutas y se especifican las operaciones de búsqueda mediante pre y postcondiciones que garantizan propiedades como la distancia máxima de acceso a las rutas y la viabilidad de los transbordos. La propuesta se valida mediante la herramienta VDM++ Toolbox, abordando así tanto la correctitud del algoritmo como la escalabilidad de la solución.**

***Index Terms*—métodos formales, VDM++, transporte público, planificación de rutas, sistemas críticos**

I. Introducción

I-A. Definición del Problema

En la ciudad de Arequipa, la información sobre las rutas del transporte público es dispersa, está desactualizada y no se encuentra disponible de forma clara para el ciudadano. Ante la necesidad de desplazarse entre dos puntos, el usuario desconoce qué línea de bus tomar, cuáles son sus paraderos y si es necesario realizar un transbordo. Esta falta de información confiable produce pérdida de tiempo, desplazamientos innecesarios y decisiones de movilidad incorrectas.

La planificación de viajes en transporte público constituye una tarea computacionalmente *crítica*: un resultado erróneo (una ruta que no pasa cerca del origen, un transbordo inviable o una dirección de recorrido incorrecta) induce al usuario a tomar una decisión equivocada, con el consiguiente costo en tiempo y seguridad. Por este motivo, la correctitud del algoritmo de búsqueda de rutas no puede garantizarse únicamente mediante pruebas de ejecución; se requiere una especificación formal que demuestre propiedades sobre el resultado producido, tal como se argumenta para los sistemas de implementación crítica [1].

El presente trabajo propone una plataforma web para la búsqueda de rutas de transporte en Arequipa. Su componente crítica, el algoritmo de búsqueda de rutas directas y combinadas, es especificada y validada formalmente mediante el lenguaje VDM++ [2], utilizando invariantes, precondiciones y postcondiciones. La validación del modelo se realiza con la herramienta VDMTools [2].

I-B. Objetivos

I-B1. Objetivo general: Desarrollar una propuesta de solución para la planificación de viajes en transporte público, cuya componente crítica, el algoritmo de búsqueda de rutas, sea especificada y validada formalmente con VDM++, garantizando su correctitud y escalabilidad.

I-B2. Objetivos específicos:

1. Definir los tipos de datos e invariantes del dominio de rutas (recorridos, paraderos y radios de búsqueda).
2. Especificar en VDM++ las operaciones de búsqueda de rutas directas y de rutas combinadas mediante pre y postcondiciones.
3. Establecer las propiedades que el algoritmo debe garantizar, tales como la distancia máxima de acceso a una ruta, la viabilidad de los transbordos y la longitud mínima de los segmentos.
4. Validar el modelo formal con la herramienta VDM++ Toolbox mediante obligaciones de prueba y casos de prueba.
5. Documentar los requerimientos funcionales y los trabajos relacionados que sustentan la propuesta.

II. Trabajos Relacionados

La presente propuesta se fundamenta en la línea de investigación de la especificación formal con VDM++ y su aplicación a sistemas de implementación crítica y de gran escala.

Respecto al soporte de herramientas, Fitzgerald, Larsen y Sahara [2] describen VDMTools, un conjunto de herramientas que analizan modelos expresados en VDM y sus dialectos VDM++ y VDM-RT, y que ofrecen verificación estática, generación automática de código, mapeo a UML, análisis de cobertura de pruebas y depuración. Este trabajo destaca que VDMTools está orientado al desarrollo rentable de modelos formales en entornos industriales, lo que sustenta la elección de esta herramienta para validar el algoritmo propuesto.

En el ámbito de los sistemas críticos, Verhoef, Larsen y Hooman [3] extienden VDM++ para modelar y validar sistemas embebidos distribuidos en tiempo real, definiendo una semántica operacional constructiva validada con VDMTools. De forma complementaria, Verhoef [1] analiza el uso de VDM++ para especificar sistemas de tiempo real, argumentando que los métodos formales deben integrarse a los flujos de trabajo

industriales y permitir el análisis temprano de propiedades. Ambos trabajos justifican la aplicación de VDM++ a un componente cuya correctitud es crítica, como el algoritmo de planificación de viajes propuesto.

Finalmente, Mochio y Araki [4] vinculan VDM++ con el desarrollo formal ágil y escalable de software de misión crítica, incluyendo la descripción de arquitecturas y la verificación de especificaciones. Este enfoque respalda el requisito de escalabilidad de la solución, en la medida en que el modelo formal puede acompañar la evolución del sistema.

A diferencia de los trabajos citados, centrados en sistemas de tiempo real o en la infraestructura de herramientas, esta propuesta aplica VDM++ a la verificación de un algoritmo de planificación de rutas de transporte público, en el contexto específico de la ciudad de Arequipa.

III. Requerimientos Funcionales

La propuesta plantea una plataforma web con dos funcionalidades principales: la búsqueda de rutas de transporte y el reporte colaborativo de incidencias urbanas. Los requerimientos funcionales que se describen a continuación constituyen la base de la especificación formal; los requerimientos RF1 y RF2 son los que serán especificados y verificados con VDM++ en el siguiente entregable.

Cuadro I
Requerimientos funcionales del sistema.

ID	Requerimiento funcional
RF1	Búsqueda de rutas directas: para un origen y un destino dados, el sistema debe devolver las rutas cuyo recorrido pase a una distancia menor o igual a un radio definido (1000 m) de ambos puntos.
RF2	Búsqueda de rutas combinadas: el sistema debe devolver combinaciones de dos rutas con un punto de transbordo viable, respetando una distancia máxima de transbordo (100 m) y una longitud mínima de segmento (500 m).
RF3	Visualización en mapa: el sistema debe mostrar en un mapa interactivo el recorrido de las rutas y sus paraderos.
RF4	Estimación de distancia y tiempo: el sistema debe calcular la distancia total del viaje y un tiempo estimado, incluyendo el tiempo de transbordo.
RF5	Gestión de datos de rutas: el sistema debe permitir cargar rutas desde archivos KML y asociar los paraderos cercanos a cada recorrido.

Los requerimientos RF1 y RF2 se formalizarán en VDM++ definiendo los tipos de datos (puntos, recorridos), las invariantes del dominio (longitud mínima del recorrido, validez de coordenadas) y las operaciones de búsqueda con sus correspondientes pre y postcondiciones. Por ejemplo, la postcondición asociada a RF1 establece que toda ruta directa devuelta debe encontrarse a una distancia menor o igual al radio definido tanto del origen como del destino.

IV. Modelo en VDM++

El componente crítico del sistema, el algoritmo de búsqueda de rutas, se especificó como una clase VDM++ denominada

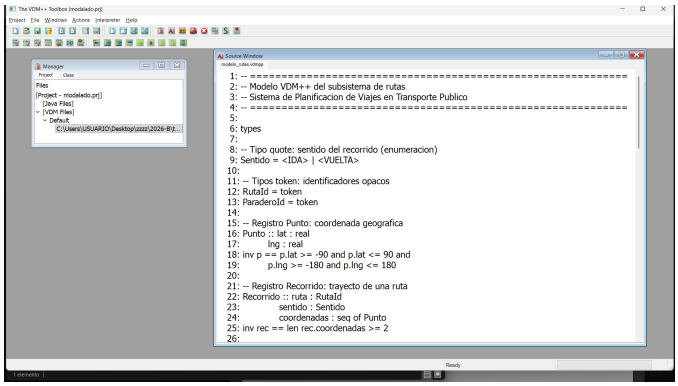


Figura 1. Modelo VDM++ del subsistema de rutas cargado en la VDM++ Toolbox.

`Sistema`. El modelo se organiza en tipos, variables de instancia, funciones auxiliares y operaciones con pre y postcondiciones, y cubre los elementos del lenguaje requeridos: tipos *quote*, *token*, registros con invariantes, secuencias, conjuntos y mapas. La Figura 1 muestra el modelo cargado en la VDM++ Toolbox.

IV-A. Tipos e invariantes

La especificación define un tipo *quote* para el sentido del recorrido, dos tipos *token* para los identificadores opacos, y dos tipos registro con sus invariantes:

```
class Sistema
types
  public Sentido    = <IDA> | <VUELTA>;      -- quote
  public RutaId     = token;                 -- token
  public ParaderoId = token;                 -- token

  public Punto :: lat : real                 -- record
                  lng : real
  inv p == p.lat >= -90 and p.lat <= 90 and
           p.lng >= -180 and p.lng <= 180;

  public Recorrido :: ruta : RutaId          -- record
              sentido : Sentido
              coordenadas : seq of Punto
  inv r == len r.coordenadas >= 2;

instance variables
  recorridos : set of Recorrido := {};
  paraderos  : set of ParaderoId := {};
```

Los tipos *quote* representan valores simbólicos de un dominio cerrado (ida o vuelta). Los tipos *token* representan identificadores opacos que solo admiten igualdad, por lo que no es necesario conocer su estructura interna. Los registros agrupan campos con nombre y sus invariantes restringen los valores válidos: las coordenadas deben estar en el rango geográfico ($-90° \leq lat \leq 90°$ y $-180° \leq lng \leq 180°$) y todo recorrido debe tener al menos dos puntos, pues un trayecto requiere origen y destino.

IV-B. Colecciones: secuencia, conjunto y mapa

El modelo usa tanto secuencias como conjuntos. Las **secuencias** (`seq of Punto`) se emplean para las coordenadas,

porque el orden de los puntos define el trayecto y un punto puede repetirse en una parada. Los **conjuntos** (`set of Recorrido`, `set of ParaderoId`) se emplean para los catálogos, donde el orden no importa y no debe haber duplicados. Adicionalmente, se propone un **mapa** para relacionar cada ruta con sus recorridos:

```
recorridosPorRuta : map RutaId to set of Recorrido
inv recorridosPorRuta ==
  (forall r in set dom recorridosPorRuta &
     card recorridosPorRuta(r) <= 2)
  and
  (forall r in set dom recorridosPorRuta &
     forall rec in set recorridosPorRuta(r) & rec.ruta = r)
     ;
```

La invariante del mapa expresa que cada ruta tiene a lo sumo dos recorridos (ida y vuelta) y que cada recorrido del mapa referencia a la ruta que lo contiene.

IV-C. Implementación validada

Por limitaciones de la versión de la Toolbox empleada (VDMTools 9.0.7), la implementación ejecutable representa los tipos registro como tipos producto y los identificadores *token* como naturales, preservando la semántica y las invariantes del modelo:

```
types
  public Sentido   = <IDA> | <VUELTA>;
  public RutaId    = nat;
  public Punto     = real * real
    inv p == p.#1 >= -90 and p.#1 <= 90 and
             p.#2 >= -180 and p.#2 <= 180;
  public Recorrido = RutaId * Sentido * seq of Punto
    inv r == len r.#3 >= 2;
```

IV-D. Operaciones y postcondiciones

Sobre la variable de instancia `recorridos : set of Recorrido` se definen las operaciones de búsqueda. `buscarRutasDirectas` (RF1) devuelve los recorridos que pasan a una distancia menor o igual a `radio` del origen `A` y del destino `B` (en la implementación con tuplas, el campo `coordenadas` se accede como `rec.#3`):

```
public buscarRutasDirectas : Punto * Punto * real ==> set
    of Recorrido
buscarRutasDirectas(A, B, radio) ==
  return { rec | rec in set recorridos &
       cercaniaPunto(A, rec.#3) <= radio and
       cercaniaPunto(B, rec.#3) <= radio }
pre A <> B
post forall rec in set RESULT &
       cercaniaPunto(A, rec.#3) <= radio and
       cercaniaPunto(B, rec.#3) <= radio;
```

La postcondición formaliza la propiedad central: toda ruta devuelta se encuentra a una distancia menor o igual al radio respecto del origen y del destino. La operación `buscarRutasCombinadas` (RF2) extiende la búsqueda a pares de recorridos conectados por un transbordo; su postcondición exige además que las rutas sean distintas y que exista un punto de transbordo dentro de `radioTransbordo`.

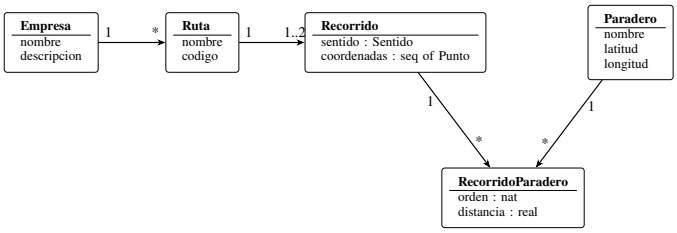


Figura 2. Diagrama de clases del dominio de rutas.

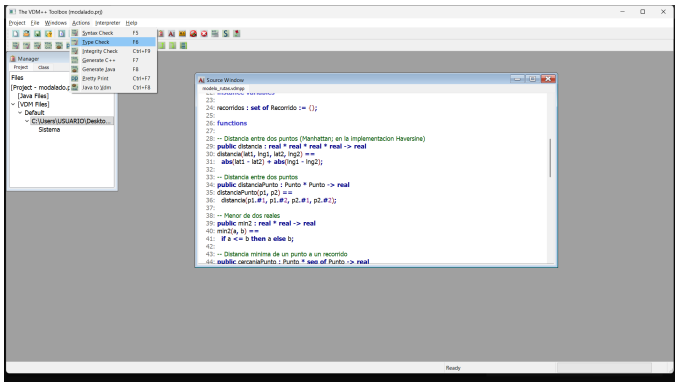


Figura 3. Análisis de tipos (*type check*) del modelo sin errores.

IV-E. Diagrama de clases

La Figura 2 presenta el diagrama de clases del dominio de rutas. Una empresa posee varias rutas; cada ruta tiene uno o dos recorridos (ida y vuelta); y cada recorrido se asocia con paraderos a través de la clase `RecorridoParadero`, que registra el orden y la distancia.

El diagrama también se documenta en formato PlantUML, lo que permite regenerarlo o exportarlo a otros formatos:

```
@startuml
class Empresa { nombre; descripcion }
class Ruta { nombre; codigo }
class Recorrido { sentido; coordenadas }
class Paradero { nombre; latitud; longitud }
class RecorridoParadero { orden; distancia }
Empresa "1" --> "*" Ruta
Ruta "1" --> "1..2" Recorrido
Recorrido "1" --> "*" RecorridoParadero
Paradero "1" --> "*" RecorridoParadero
@enduml
```

La Figura 3 muestra que el análisis de tipos del modelo finaliza sin errores.

V. Pruebas

La validación del modelo se realizó ejecutando casos de prueba en el intérprete de la VDM++ Toolbox. Los casos cubren las operaciones de búsqueda y combinan resultados esperados positivos y negativos, usando las coordenadas de la Plaza de Armas como origen y del Terminal Terrestre como destino. La Tabla II resume los casos ejecutados.

En el caso positivo, el recorrido contiene puntos exactamente en el origen y en el destino, por lo que la distancia mínima es nula y el recorrido se devuelve. En el caso negativo se utiliza

Cuadro II
CASOS DE PRUEBA EJECUTADOS EN EL INTÉRPRETE.

Caso	Entrada	Resultado
P1	Origen y destino sobre el recorrido, radio 0.05	Devuelve la ruta
N1	Origen lejano, radio 0.05	Conjunto vacío
C1	Dos recorridos con transbordo	Par de rutas

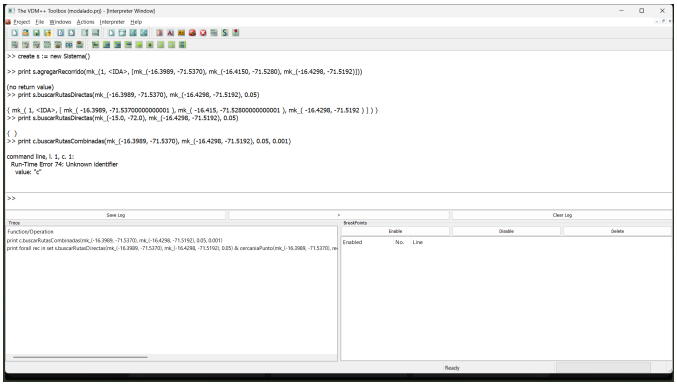


Figura 4. Ejecución de los casos de prueba en el intérprete de la Toolbox.

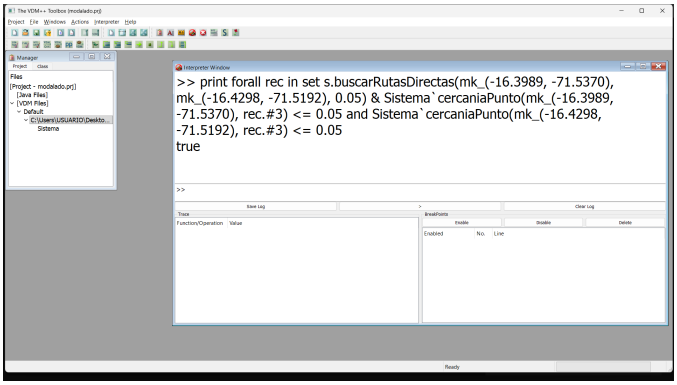


Figura 5. Evaluación de la postcondición sobre el resultado: devuelve `true`.

un origen lejano, de modo que ningún recorrido cumple la restricción de distancia y el resultado es el conjunto vacío. En el caso combinado, dos recorridos comparten un punto de transbordo dentro del radio establecido.

La Figura 4 muestra la ejecución de estos casos en el intérprete, con los recorridos construidos como tuplas y el sentido como valor *quote* (`<IDA>`).

VI. RESULTADOS

La validación se centró en comprobar que las operaciones cumplen sus postcondiciones. Además de ejecutar los casos de prueba, se evaluó directamente la postcondición de `buscarRutasDirectas` sobre el resultado obtenido. La expresión evaluada devuelve `true`, lo que confirma que todos los recorridos devueltos se encuentran dentro del radio respecto del origen y del destino (Figura 5).

Adicionalmente, la Toolbox generó las obligaciones de prueba (*proof obligations*) asociadas a las postcondiciones de

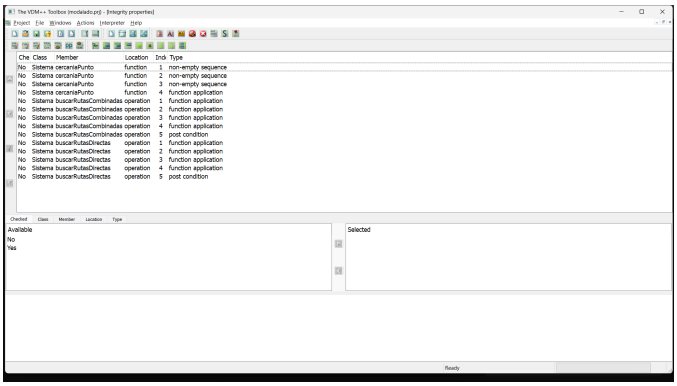


Figura 6. Obligaciones de prueba generadas a partir de las postcondiciones.

las operaciones de búsqueda (Figura 6). Estas obligaciones expresan formalmente las propiedades que el modelo debe satisfacer y constituyen la base para una verificación deductiva más estricta.

VII. DISCUSIÓN

Los resultados muestran que la especificación formal captura correctamente las propiedades centrales del algoritmo de búsqueda. Las postcondiciones de `buscarRutasDirectas` y `buscarRutasCombinadas` expresan con precisión las garantías del sistema: proximidad de las rutas al origen y al destino, y viabilidad de los transbordos. La ejecución de los casos de prueba y la evaluación directa de la postcondición confirman que el modelo satisface dichas propiedades para los datos considerados.

Sin embargo, la validación por interpretación no equivale a una demostración deductiva completa: los casos de prueba cubren escenarios representativos, pero no prueban la postcondición para todas las entradas posibles. Las obligaciones de prueba generadas por la Toolbox constituyen el punto de partida para una verificación más rigurosa.

Desde el punto de vista del modelado, la herramienta impuso algunas restricciones prácticas. En particular, los tipos registro se representaron como tipos producto (tuplas) y la distancia se modeló de forma simplificada; ambos aspectos se detallan en la Sección IX.

VIII. CONCLUSIONES

Se especificó y validó formalmente, mediante VDM++, el componente crítico de una plataforma de planificación de viajes en transporte público para Arequipa. El modelo define los tipos del dominio, las operaciones de búsqueda de rutas directas y combinadas, y sus postcondiciones, que formalizan las garantías del algoritmo.

La validación en la VDM++ Toolbox permitió comprobar que el modelo es sintáctica y semánticamente consistente, que las operaciones se ejecutan correctamente sobre casos de prueba representativos y que las postcondiciones se satisfacen. De este modo, se evidenció que aplicar métodos formales a un algoritmo de movilidad permite especificar con precisión

su comportamiento y detectar ambigüedades que las pruebas informales no revelan.

IX. Limitaciones

El trabajo presenta las siguientes limitaciones. Primero, la función de distancia se modeló de forma simplificada (distancia Manhattan sobre coordenadas), mientras que la implementación real utiliza la fórmula de Haversine; esto no altera la estructura de las propiedades verificadas, pero sí la exactitud numérica. Segundo, por compatibilidad con la versión de la herramienta, los tipos registro se representaron como tipos producto (tuplas), lo que reduce la legibilidad del modelo frente a una definición con campos con nombre. Tercero, la validación se basó en interpretación y casos de prueba, no en una demostración deductiva completa de las obligaciones de prueba. Finalmente, el conjunto de datos de prueba es reducido y no cubre toda la variabilidad de las rutas reales.

X. Recomendaciones y Trabajos Futuros

Como trabajo futuro se propone: (i) incorporar la fórmula de Haversine en el modelo para reflejar con exactitud las distancias reales; (ii) completar la verificación demostrando formalmente las obligaciones de prueba generadas por la Toolbox; (iii) ampliar el modelo con operaciones de cálculo de tiempo estimado y de ordenamiento de resultados; (iv) integrar el modelo formal con la implementación web (Django y React) para garantizar la consistencia entre especificación e implementación; y (v) ampliar el conjunto de casos de prueba con datos reales de las rutas de Arequipa, incluyendo escenarios de transbordo múltiple y geometrías complejas.

Referencias

[1] M. Verhoef, "On the use of VDM++ for specifying real-time systems," Radboud University Nijmegen, Tech. Rep. ICIS-R05033, 2005.

[2] J. S. Fitzgerald, P. G. Larsen, and S. Sahara, "VDMTools: Advances in support for formal modeling in VDM," *ACM SIGPLAN Notices*, vol. 43, no. 2, pp. 3–11, 2008.

[3] M. Verhoef, P. G. Larsen, and J. Hooman, "Modeling and validating distributed embedded real-time systems with VDM++," in *FM 2006: Formal Methods, 14th International Symposium on Formal Methods, Proceedings*, ser. Lecture Notes in Computer Science, vol. 4085. Springer, 2006, pp. 147–162.

[4] H. Mochio and K. Araki, "VDM++ as a basis of scalable agile formal software development," in *Proceedings of the 9th Overture Workshop*, 2011, pp. 50–58.